

User Guide Java SUMO

The following document represents a user manual for the Java-based SUMO application by Group3_Group4.



Starting the program

To launch the program, either use command line tools for execution of the program or press the „run“ button in an IDE of choice while having the Launcher class opened.

The program is going to start in fullscreen mode by default.



Program features

1) Navigation of the map

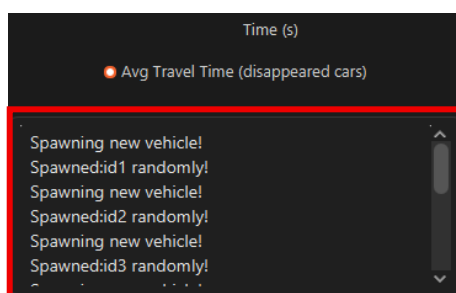
The program allows for moving around the map by left clicking on it and dragging it around the box (on success the cursor changes to following symbol: )

2) Inserting a Car



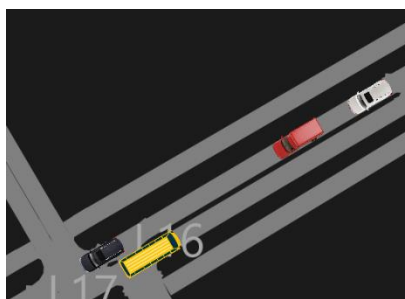
Pressing the blue button spawns a new vehicle in the simulation randomly (without choice of lane, Colour or vehicle type)

On success, an indicator sentences are shown on the text window in the bottom-right



Vehicles spawned randomly will appear on a random lane in a random combination of type (CAR, TRUCK, BUS) X colour (RED, BLACK, WHITE, YELLOW)

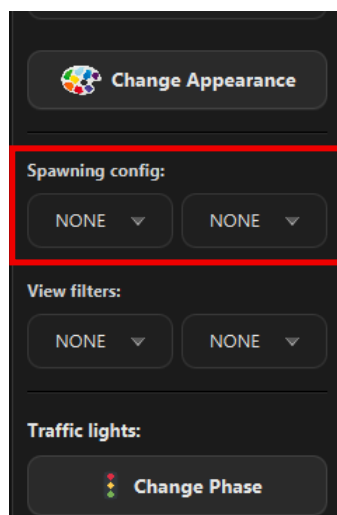
Example: A yellow bus, red truck, black car, and a white car



By clicking below the blue „Insert Car“ button, one can choose from a drop-down set of lanes to spawn vehicles on.

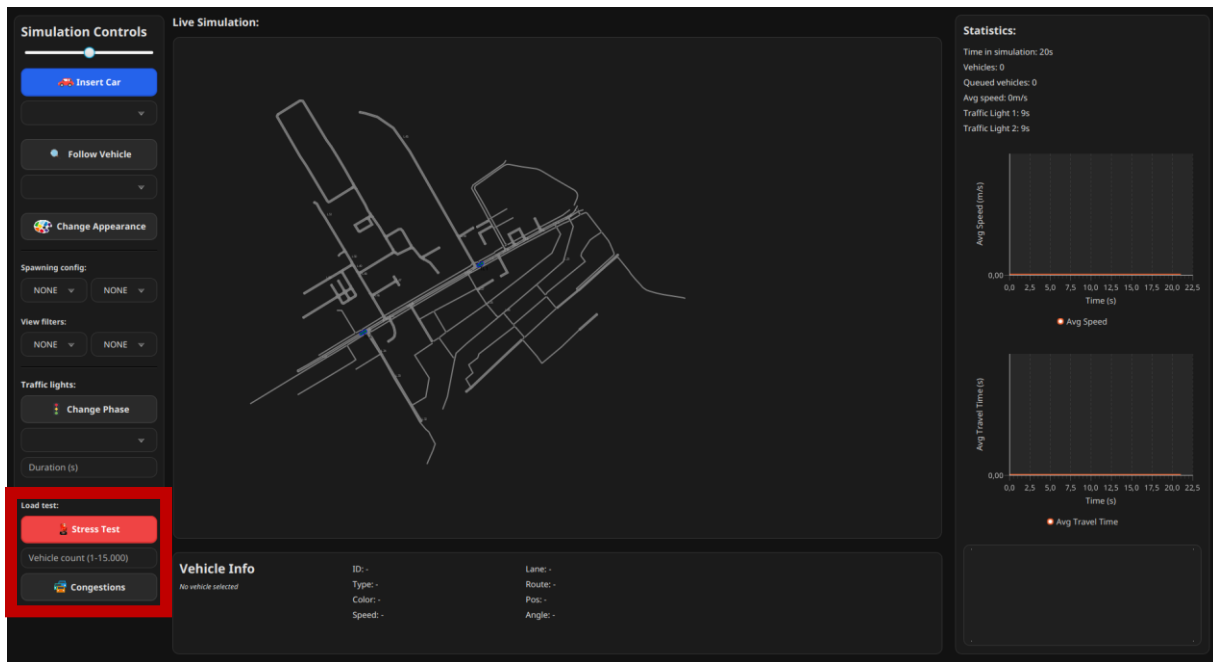


Spawning filters for type and colour can be applied in drop-down menus below the button „Change Appearance“



3) Stress Test

In order to batch-spawn vehicles, one can enter a number from 1 to 15.000 in the Stress test section.



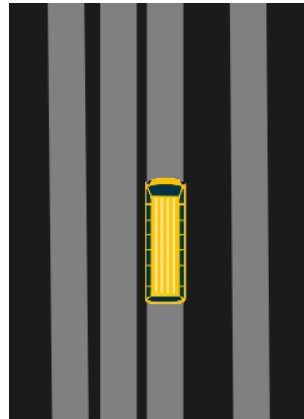
The stress test is type- and colour-sensitive. However, in order to keep the simulation from jamming, lanes cannot be chosen specifically for a stress test.

4) Following a vehicle

Upon vehicle selection via a drop-down menu, the user can follow a vehicle by first selecting its ID and then pressing the button „Follow Vehicle“



Pressing the button will center the camera on the chosen vehicle and follow it.

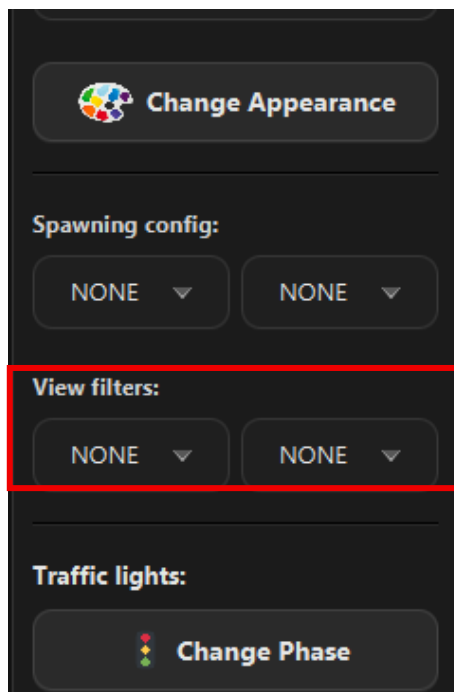


5) Changing a vehicle's appearance

The spawning filters have a secondary function as well. If a user clicks on a vehicle, it gets selected (shown in the lower box of the simulation). Then, by pressing the “Change Appearance” button, it will apply the selected spawning filters to the chosen vehicle. If no filter is selected, the car remains unchanged. If the user clicks anywhere in the map, the vehicle gets deselected.

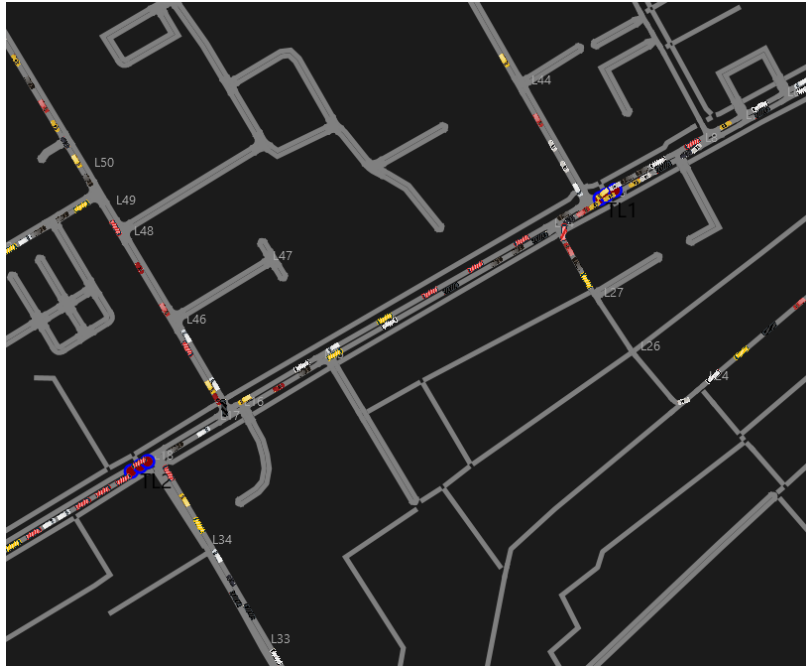
6) Filtering for vehicles

Under the section „View filters“, the user is able to select for type and colour of shown vehicles, thus turning an otherwise crowded simulation in a more manageable one.



The default for filtering is “NONE” for both the type and colour filtering

- No Filters:

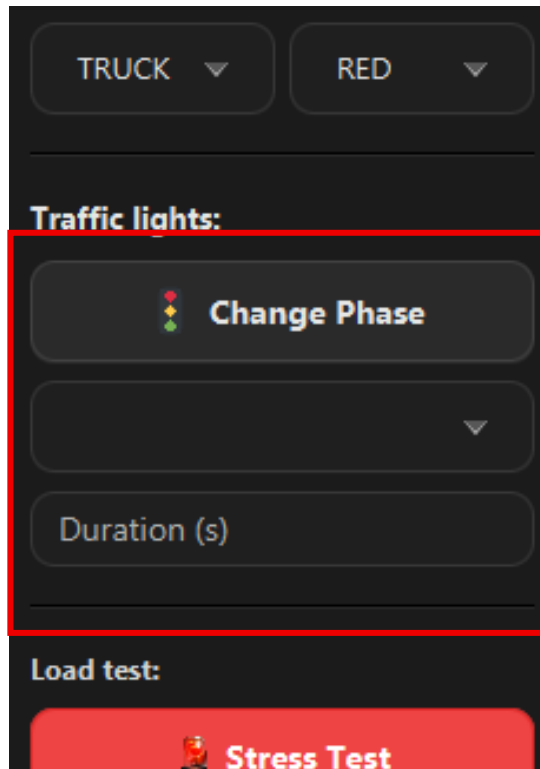


- Only Red Trucks:



7) Traffic light adjustment

The user has two groups of traffic lights to play with (each consisting of 3 traffic lights)



The screenshot shows a control panel with a dark background. At the top, there are two dropdown menus labeled 'TRUCK' and 'RED'. Below them, the section 'Traffic lights:' is highlighted with a red border. Inside this section, there is a 'Change Phase' button with a traffic light icon, a dropdown menu, and a 'Duration (s)' input field. Below the 'Traffic lights:' section is a 'Load test:' section with a red button labeled 'Stress Test'.

For this, the user has to select a traffic light group s/he prefers to change as well as typing in the duration in seconds in order to change it.

Traffic lights can only be changed to their opposing state (green -> red, red -> green)

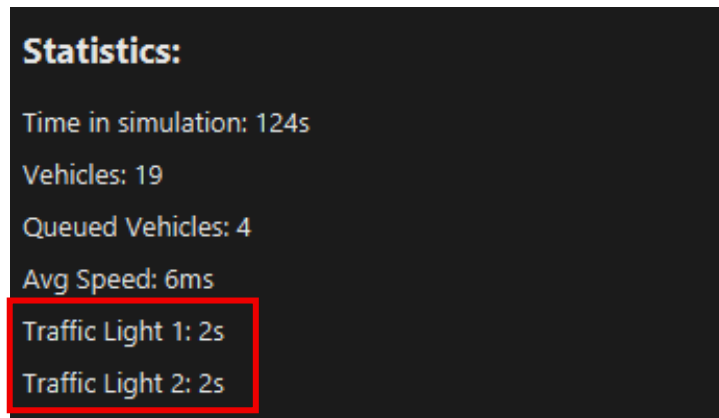
If a change of phase has been submitted, that phase duration also counts toward the duration of the opposite state (not affecting the 3s intermediate yellow state)

Example:

In this scenario, the traffic light 2 would turn to a 5 second red phase, keeping that 5 seconds of phase duration for future phase changes.



The remaining time of Traffic light phases before changing their state is shown in the statistics section



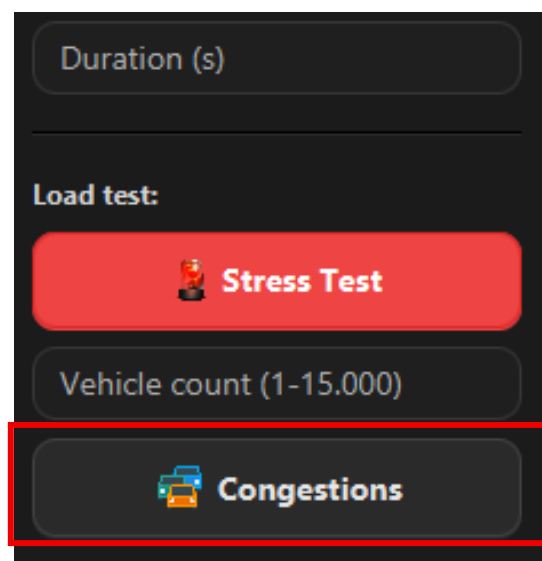
Note:

The simulation starts with a default duration of 30 seconds for the traffic lights in states green and red.

If a traffic light changes from green to red, an intermediate state, yellow, is triggered for 3 seconds. The intermediate state of yellow does not apply to the change from red to yellow.

8) Congestion Detection

A congestion in the simulation occurs if the average speed of vehicles on a current lane is below 1 m/s (negative speed cannot occur)



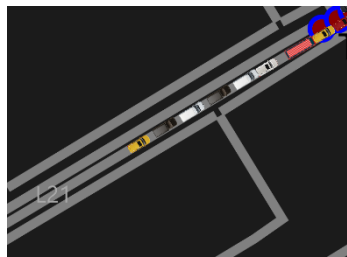
If no congestion is present, the indicator sentence “No congestion found“ is printed on the text window.


```
No congestion found
No congestion found
No congestion found
```

Example:

```
Congestion detected:
Lane L28 → avg speed: 0,00 m/s
Lane -L53 → avg speed: 0,00 m/s
Lane -L52 → avg speed: 0,00 m/s
Lane L21 → avg speed: 0,01 m/s
Lane -L9 → avg speed: 0,00 m/s
```

Lane L21 is congested with an average speed of 0.01 m/s.



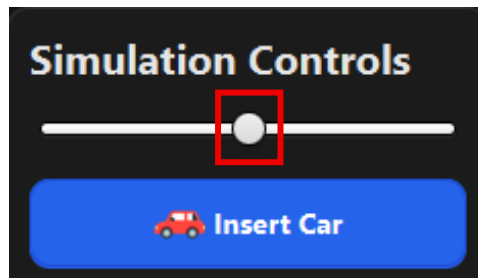
Note:

A congestion can best be forced by triggering multiple stress tests and altering the phase of traffic lights to a long time of phase red.

Negative lanes are those that go in the opposite direction of their positive equivalent.

9) Altering the speed of the simulation

The user can speed up the simulation by moving the bar along the right side. To slow it down, the user has to move it to the left.



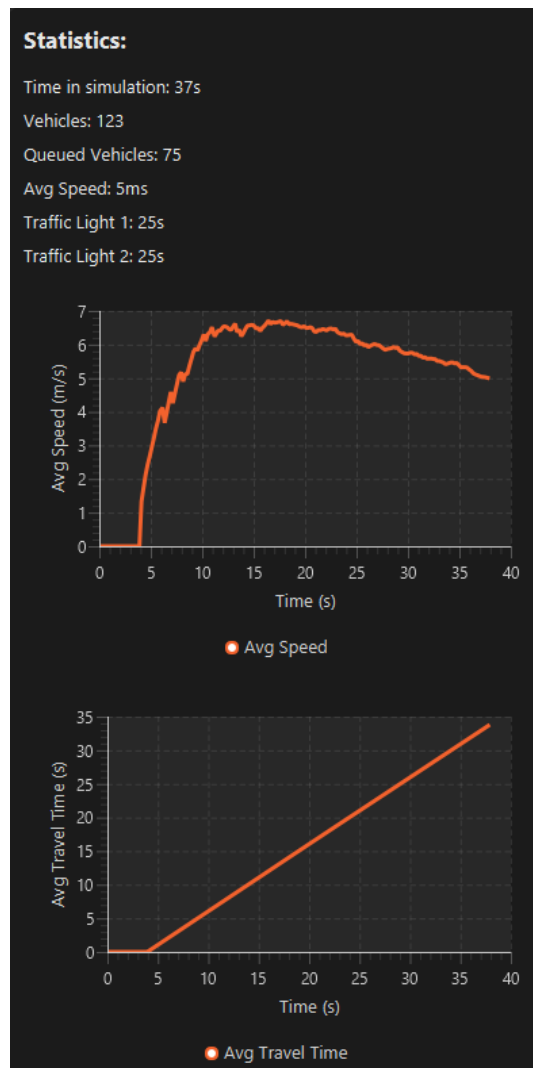
10) Vehicle info panel

By hovering over a vehicle that's currently in the simulation, the user can retrieve information about the individual vehicle's current state.

Vehicle Info		
Selected: id1	ID: id1	Lane: L17
	Type: car	Route: r18
	Color: YELLOW	Pos: 305,57, 336,68
	Speed: 9,99 m/s	Angle: 59

11) Statistics section

The statistics section is there to show relevant live metrics about the simulation



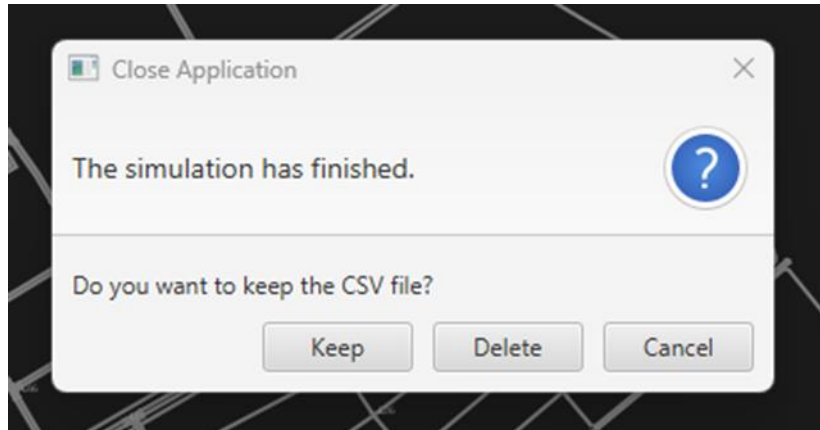
It contains:

- Time in simulation
- Amount of vehicles in the simulation
- Amount of vehicles to insert in queue
- Average speed of the vehicles in the simulation
- Times until the traffic light's next phase switch
- Chart showing the average speed of vehicles in the simulation
- Chart showing the average travel time of vehicles in the simulation
 - o $\text{Average travel time} = \text{time each vehicle has spent in simulation} / \text{\#vehicles}$

12) CSV extraction

Upon the start of the program, a CSV file called „simulation.csv“ with simulation data is created automatically and stored in the working directory.

When closing the simulation, the user is prompted to either keep or delete the file.



If a simulation.csv already exists within the working directory and the simulation is started over again, the simulation.csv is overwritten by a new file. Thus, the simulation.csv file always represents either the current simulation (while running) or the last running instance of the program.

Authors:

1. Minh Luan Vo
2. Lars Michael Keppler
3. Justin Samuel Bons
4. Ali Ajib
5. Abel Solyom